

一排点击日志，只一眼就看清

第5讲 · 集合与字典

取唯一 · 按名算账

这一讲，你会收获四件事

- 取消重复：集合一键去重、交并差
- 按名找值：字典存"键→值"账单
- 四种造法：字典怎么建，各有各的用
- AI 落地：词频统计、配置映射、共同兴趣

第一幕 · 取消重复

只收不重复的收纳盒

```
tags = {"体育", "科技", "游戏", "体育", "科技"}  
print(tags)           # {'体育', '科技', '游戏'}  
                        # └ 重复自动去掉、顺序不定
```



重复的，放一个就够

和列表、元组站一起比一比

类型	顺序	允许重复	可否下标
list	有	可以	<code>l[0]</code>
tuple	有	可以	<code>t[0]</code>
set	无	不可以	不可以

增删改，外加四种"算账"

```
s = set()           # 空集合 注意 {} 是空字典
s.add("a")          # 增,重复不生效
s.discard("b")      # 删,不存在也不报错
```

```
A = {"体育", "科技", "音乐"}
```

```
B = {"科技", "游戏"}
```

```
print(A & B)        # 交集 {科技}
```

```
print(A | B)        # 并集 {体育,科技,音乐,游戏}
```

```
print(A - B)        # 差集 {体育,音乐}
```



海量数据去重，一行见底

```
raw = ["A", "B", "A", "C", "B", "A"]
unique = sorted(set(raw))
print(unique)          # ['A', 'B', 'C']

userA = {"电子", "游戏", "运动"}
userB = {"运动", "摄影", "电子"}
print(userA & userB)    # 共同兴趣 {运动, 电子}
```



第二幕·按名算账

像电话号码本：按"姓"找"人"

```
user = {"name": "Alice", "age": 20}
print(user["name"])      # Alice   用键取值
user["age"] = 21         # 改值
user["city"] = "Xiamen"  # 新增键值对
print(user)
```



键必须不可变 · 值随便改

要名还是要用下标？ 三条路

维度	列表	元组	字典
结构	有序序列	有序序列	无序键→值映射
访问	<code>l[0]</code>	<code>t[0]</code>	<code>d["key"]</code>
键类型	任意	任意	键须可哈希
用途	按序存一批	固定只读	按名称查询

★ 一种字典，四种造法

```
d1 = dict(name="Alice", age=20)           # ① 关键字参数
d2 = {"name": "Bob", "age": 21}           # ② 字面量最直观
d3 = dict.fromkeys(["x", "y", "z"], 0)    # ③ 批量默认值
keys   = ["sport", "tech", "game"]
values = [10, 20, 30]
d4 = dict(zip(keys, values))              # ④ 两串配对
```



- 字面量 / 关键字：手写最省事
- `fromkeys`：先给一堆键，统统抄一个默认值
- `zip`：先存好两组数据，再双排配对

取值得"安全"，遍历分三路

```
d = {"sport": 10, "tech": 20}

print(d.get("sport"))           # 10
print(d.get("food", "免费默认")) # 找不到也不报错

for k in d:                     print(k)           # 键
for v in d.values():            print(v)          # 值
for k, v in d.items():          print(k, "→", v)  # 键值对
```



数一句台词，各归其位

```
words = "the apple the banana the apple".split()
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
print(freq)    # {'the':3, 'apple':2, 'banana':1}

config = {"learning_rate": 0.01, "epochs": 50,
          "optimizer": "adam", "device": "cuda"}
print(config["optimizer"])    # adam
```



第三幕·挑对容器

三个场景，三种答案

去重清洗 → 集合 set

活动日志去重, 只要唯一用户

词频统计 → 字典 dict

每个词 → 计数, 按名 +1

模型配置 → 字典 dict

"learning_rate" → 0.01, 按名读取



五个坑，踩一个算我输

想干嘛	坑	对的做法
建空集合	<code>{}</code> 是空字典	空集合用 <code>set()</code>
取集合元素	<code>s[0]</code> 没下标	用 <code>for</code> 遍历
做键/元素	列表不可哈希	数字/字符串/元组
键重复	后面盖掉前面	键要唯一
取不存在的键	报 <code>KeyError</code>	用 <code>d.get(k, 默认)</code>

结课：选对容器，数据才流得顺

要唯一用 set，要映射用 dict，要按序用 list，要只读用 tuple——四口箱子，各装各的货。



下节预告：条件结构 if / elif / else